AI RISK MANAGER

Razorpay AI Buildathon — Track 02

Defense-first AI-assisted fraud-risk detection and merchant risk management

Project blueprint and documentation for building the application in an AI app builder.

Important: this document does not invent production results. Any precision, recall, or other performance figures must be measured from the actual application and held-out test set before submission.

1. Executive Summary

AI Risk Manager is a merchant-facing application that identifies suspicious payment activity, assigns a risk score, explains the main risk signals, and recommends a safe defensive action. The first version focuses on fraud-risk detection and can later be extended to returns and chargebacks.

This matches the purpose of Razorpay's official AI Buildathon Track 02, which asks participants to build a working detector, verifier, or auto-responder for a class of merchant loss and evaluate it with precision and recall on a held-out test set. Official challenge: https://razorpay.com/buildathon/

2. Problem Statement

• Merchants can lose money through fraudulent transactions and abnormal payment activity.

• Manual review can be slow and can create unnecessary friction for legitimate customers.

• Risk decisions need to be explainable so a merchant understands why an alert was generated.

• A useful detector must balance fraud detection against the cost of false positives.

3. Proposed Solution

The application receives transaction signals such as amount, transaction velocity, payment-method pattern, device consistency, location consistency, and recent account behavior. A risk engine converts these signals into a score from 0 to 100 and classifies each transaction as Low, Medium, or High risk.

Recommended defensive actions: **Low** — allow and monitor; **Medium** — step-up verification or manual review; **High** — hold for review and create an investigation alert.

4. Core Features

• Merchant dashboard with transaction and alert summaries.

• Transaction-level risk score from 0–100.

• Explainable risk factors for every alert.

• Fraud-spike monitoring for sudden changes in suspicious activity.

• Review queue for Medium and High risk events.

• Recommended safe action and audit trail.

• Evaluation dashboard with precision, recall, false-positive rate, false-negative rate, and confusion matrix.

• Synthetic demo-data mode to avoid exposing real customer information.

5. System Architecture

Layer	Purpose
Frontend	Dashboard, transaction table, risk score, explanations and review queue

Data	Synthetic/demo transactions and optional CSV input
Risk Engine	Feature extraction, scoring and risk classification
AI Explanation	Natural-language explanation of important risk signals
Evaluation	Held-out test set, precision/recall and false-positive cost
Audit	Decision, reason, timestamp and reviewer action

6. Risk Scoring

For the prototype, use a transparent scoring model. Example signals are normalized and combined into a score. A later version can replace this with a trained ML model.

Example prototype formula:

```
Risk Score = weighted combination of normalized risk signals × 100
```

Thresholds should be selected using validation data. Do not claim that a threshold is optimal until it has been tested.

7. AI Builder Prompt

Copy this prompt into your AI builder:

```
Build a responsive web application named “AI Risk Manager” for Razorpay AI Buildathon Track 02. The app must be defense-only and focus on detecting suspicious payment activity for merchants. Create: a transaction upload/demo-data generator; transaction table; risk score from 0–100; LOW/MEDIUM/HIGH classification; explainable risk factors; recommended safe action; review queue; audit log; and an evaluation dashboard showing precision, recall, false-positive rate, false-negative rate, confusion matrix and false-positive cost. Use synthetic data for the demo. Keep the risk logic transparent and make the detector replaceable with an ML model later. Add input validation, error handling, mobile responsiveness, and a clean fintech dashboard. Do not create functionality that helps bypass fraud controls or commit fraud. Include a README with setup, architecture, assumptions, limitations and evaluation instructions.
```

8. User Flow

Merchant opens dashboard → loads synthetic transactions → system extracts risk signals → calculates risk score → explains risk factors → classifies transaction → recommends defensive action → merchant reviews alert → decision is logged → evaluation metrics are shown.

9. Dataset & Evaluation Plan

Use a synthetic labeled dataset containing legitimate and suspicious transactions. Separate the data into training/validation and a completely held-out test set. Tune thresholds only on validation data, then report final results on the held-out test set.

Metric	What it tells the reviewer
Precision	How many flagged transactions were actually risky
Recall	How many risky transactions were successfully detected
False-positive rate	How often legitimate transactions were incorrectly flagged
False-positive cost	Business impact of unnecessary review/friction
False-negative cost	Business impact of missed risky activity

10. Example Demo Scenario

Scenario: A merchant normally receives stable transaction traffic, then suddenly receives a burst of high-value transactions with repeated attempts and a new device pattern.

Expected result: The system raises the risk score, explains the unusual signals, adds the event to the review queue, and recommends verification or manual review. It should not automatically accuse a customer of fraud.

11. Build Challenges & Technical Obstacles

• Data quality — use validation and consistent feature formats.

• False positives — tune thresholds on validation data and explicitly report false-positive cost.

• Explainability — show the main signals contributing to each alert.

• Evaluation leakage — keep the final test set untouched until evaluation.

• Reliability — validate inputs and provide a safe manual-review fallback.

• Privacy — use synthetic or anonymized data in the demonstration.

12. Suggested Final Form Answers

Selected Track: Track 2: AI Risk Manager

Project Name / Title: AI Risk Manager

Project Objectives: AI Risk Manager helps merchants detect and prioritize suspicious payment activity, understand the reasons behind risk alerts, and take safe defensive actions such as monitoring, verification, or manual review. The goal is to reduce financial losses while measuring and controlling false positives.

Build Challenges & Technical Obstacles: The main challenges were designing reliable risk signals, reducing false positives, explaining risk decisions, and evaluating the detector fairly. I addressed these by using transparent scoring, validation-based threshold selection, synthetic test data, and a held-out evaluation set with precision, recall, and false-positive cost.

13. Submission Checklist

• Public GitHub repository containing the actual working project.

• README containing problem, solution, setup, architecture, assumptions and limitations.

• 5-minute pitch video demonstrating the working flow.

• Architecture diagram.

• Held-out test-set evaluation with honest precision and recall.

• False-positive cost and threshold discussion.

• Defense-only behavior.

14. Final Note

Razorpay's challenge page specifically emphasizes a working detector and measured precision/recall, including false-positive cost. Therefore, use this PDF as your project plan/documentation, but complete the actual AI Builder application and run the evaluation before submitting the final form.

Razorpay also documents test mode for development, where test transactions use dummy data and do not involve real money: https://razorpay.com/docs/x/quickstart/